

Spatial Data Quality, a field guide

Four checks, in Python and R, before you trust a shapefile or GeoJSON — what each one catches, and the call that runs it. Keep this next to your next QA pass.

01 INSPECT — WHAT AM I EVEN HOLDING?

Features	How many rows. Decides whether a slower, pairwise check (topology) is even practical to run.
Geometry	Point, Line, Polygon, or a mix — some checks only apply to one family.
CRS	The coordinate reference system the file claims to be in — not yet verified against its own coordinates (that's step 04).
Bounds	The bounding box, in the file's own CRS units. A box in the thousands or millions is usually the first hint of a CRS problem.
Attributes	How many non-geometry columns, and their names.
Invalid	Geometries failing the Simple Features validity rules — see step 02.
Empty	Geometries with no coordinates at all — easy to miss in a raw file.
Duplicates	Exact repeats, by comparing binary geometry — not a full topological equality check (that's a slower, separate question).

→ SAME EIGHT NUMBERS, BOTH LANGUAGES

PYTHON

```
len(gdf)                # feature count
gdf.geometry.geom_type  # per-feature geometry type
gdf.crs.to_string()     # CRS
gdf.total_bounds        # [minx, miny, maxx, maxy]
gdf.geometry.to_wkb().duplicated() # exact-match duplicates
```

R

```
nrow(data)              # feature count
sf::st_geometry_type(data) # per-feature geometry type
sf::st_crs(data)$input   # CRS
sf::st_bbox(data)        # xmin, ymin, xmax, ymax
dup_key <- sf::st_as_text(sf::st_geometry(data))
duplicated(dup_key)      # exact-match duplicates
```

A summary is a starting point, not an endpoint — it tells you which of the other three checks you actually need next, not whether the file is "good."

02 VALIDATE — IS EACH SHAPE WELL-FORMED?

● Self-intersection

The boundary crosses itself — a "bowtie" instead of one closed area. The most common case, often from imprecise digitizing.

Spot it: `geom.is_valid` → `False`, reason mentions **"Self-intersection"**

● Hole outside boundary

An inner ring (e.g. a lake) extends even partly outside the outer boundary it's supposed to sit inside.

Spot it: reason mentions **"Interior is disconnected"** or **"Hole lies outside shell"**

● Self-touching ring

The boundary touches itself at a single point without crossing — splits the shape into two lobes joined by one vertex.

Spot it: reason mentions **"Ring Self-intersection"**

● Overlapping holes

Two inner rings of the same polygon intersect each other — the "no-go" area is ambiguous where they overlap.

Spot it: reason mentions **"Interior is disconnected"**

→ CHECK & FIX, SAME THREE MOVES IN BOTH LANGUAGES

PYTHON

```
geom.is_valid          # True/False
explain_validity(geom) # why, in words
make_valid(geom)       # rebuild it valid
```

R

```
sf::st_is_valid(x)          # TRUE/FALSE
sf::st_is_valid(x, reason = TRUE) # why, in words
sf::st_make_valid(x)        # rebuild it valid
```

Diagnosis first, repair second — `make_valid()` / `st_make_valid()` can change a shape's area or split it into multiple parts. Always check **what** changed before trusting the repaired geometry downstream.

GO DEEPER [Full walkthrough \(Python + R\) →](#) [Try the Validator tool →](#)

03 CHECK TOPOLOGY — DO THE SHAPES RELATE TO EACH OTHER CORRECTLY?

● Overlap

Two polygons that should only be adjacent instead cover some of the same surface — common after a manual edit or a bad merge.

Spot it: `geom1.overlaps(geom2) → TRUE`, then measure `geom1.intersection(geom2).area`

● Gap

A sliver of uncovered space is left between polygons that should continuously tile an area, with no feature to claim it.

Spot it: for close, non-overlapping pairs, `union(a,b).convex_hull.difference(union(a,b))` is non-empty

● Sliver

A thin, geographically meaningless leftover polygon — usually the byproduct of an overlay or intersection operation.

Spot it: compactness $4 \cdot \pi \cdot \text{area} / \text{perimeter}^2$ close to 0 (a circle scores 1)

● Duplicates

Two features carry the exact same geometry — inflates counts and areas silently in any aggregation downstream.

Spot it: `geom1.equals(geom2) → TRUE` (polygons or lines alike)

● Dangle

A line endpoint that should connect to another line (a road network, a river network) stays isolated instead.

Spot it: an endpoint with no other line endpoint within tolerance distance

→ SAME IDIOMS, R SIDE

R

```
sf::st_overlaps(a, b)           # logical matrix
sf::st_equals(a, b)             # exact-geometry duplicates
sf::st_union(sf::st_combine(x)) # dissolve, for gap-hunting
```

None of these five are covered by `is_valid` — a topologically broken dataset can be made entirely of individually-valid geometries. Validity and topology are two separate questions, checked separately.

GO DEEPER [Full walkthrough \(Python + R\) →](#) [Try the Topology Checker →](#)

04 CRS — IS IT EVEN IN THE RIGHT PLACE?

● Geographic CRS

Degrees — roughly -180..180 longitude, -90..90 latitude. WGS84 (EPSG:4326) is the common one. GeoJSON and MapLibre always assume this.

Spot it: `crs.is_geographic` → `True` , coordinates inside that range

● Projected CRS

Linear units, usually meters — UTM zones, Web Mercator, national grids. Coordinates routinely in the hundreds of thousands or millions.

Spot it: `crs.is_geographic` → `False` , coordinates far outside $\pm 180/\pm 90$

The mismatch, in one line: a file can declare one kind of CRS while its actual coordinates look like the other. That disagreement is the single most common reason a map "looks wrong" before you've touched a single geometry — and neither validity (02) nor topology (03) would ever catch it, since both only look at shape, never at where it sits.

→ DETECT, THEN CONVERT — TWO SEPARATE MOVES

PYTHON

```
gdf.crs.is_geographic      # geographic or projected?
gdf.to_crs("EPSG:3857")    # reproject (only once the CRS is RIGHT)
```

R

```
sf::st_is_longlat(data)    # geographic or projected?
sf::st_transform(data, 3857) # reproject (only once the CRS is RIGHT)
```

Converting only makes sense once the **declared** CRS is actually correct — `to_crs()` / `st_transform()` mathematically transforms numbers that were never really in that CRS to begin with if the label itself is wrong. Fixing a wrong label is a judgment call only the data's own origin can make; no tool can automate it safely.

Try all four on your own file — free, runs in your browser

Upload a GeoJSON or shapefile and get the full diagnosis instantly, in Python and R side by side. Nothing leaves your computer.

[Inspector](#) → [Validator](#) → [Topology Checker](#) → [CRS Inspector](#) →